

AI for Economic Research: Dynamic Models, Language, and Agents

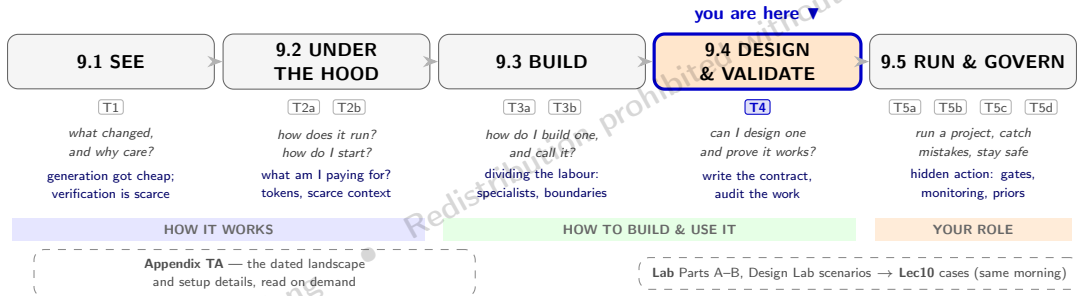
Lecture 9.4: Design Lab — Your Agent for an Economics Task

● Zhigang Feng

2026-09-12 20:17:39

Lecture 9 in One Map: Design Lab

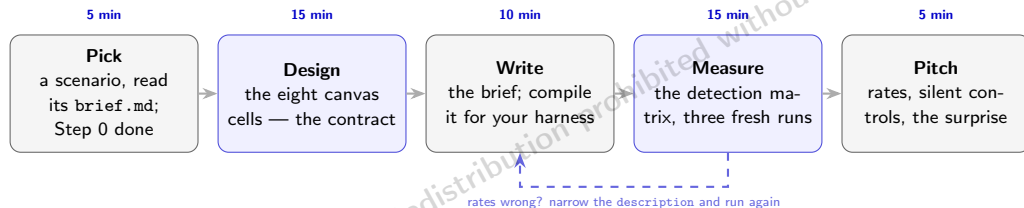
This part answers: can you design an agent for a question of your own — and prove it works?



Route: pick a scenario → fill the canvas → write the brief and compile it for your harness → run the detection matrix, three times → pitch what you measured. The economist's question for this part: how do you write the contract — and audit it?

The Design Lab in One Slide

Fifty minutes, teams of two to four, at least one working harness per team: design one agent, then measure it — three times — before you believe it.



Deliverables

- the filled canvas and the brief it compiles to
- one matrix row per agent, as rates over three runs
- one sentence on the cell that surprised you

The economist's version: the canvas is the contract, the matrix is the audit, and three runs are the sample. A reviewer you have not measured is one you are trusting for no reason.

Scenario Menu

Three scenarios, each built so a careful agent can be caught being careless — and a careless one caught being agreeable.

Scenario	The task	What it tests	Materials
S1 Aiyagari solver audit (<i>guided start</i>)	review a solver that runs and prints plausible numbers	silent numerical failure; telling a stated scope from a bug	review_target/, six lab agents
S2 A leading question	"explain how the 2022–23 rate hikes brought inflation down"	a leading verb on real data; what would identify the effect	FRED FEDFUNDS, CPIAUCSL; analysis.py
S3 Fellows provenance	audit ten rows of the Econometric Society Fellows dataset	sourcing, missingness, and a gate no agent can talk past	the Lecture 10 pilot batch and its rules
S4 Referee: skill or agent? (<i>discussion</i>)	should a referee workflow be a skill or a sub-agent?	where the reasoning lives (T3a)	Lec10's review-paper skill

Every scenario runs on Claude Code, Gemini CLI, or Codex CLI through scripts/detection_matrix.py. Answer keys live in the lab's instructor/ folder — do not open it before you pitch.

S1 Brief — Audit the Aiyagari Solver

The solver runs to completion and prints plausible numbers. Design the reviewer you would want on it before a result goes into a paper.

The situation

- review_target/aiyagari_solver.py: value-function iteration for the Aiyagari household problem, annual calibration
- six lab agents already exist — three by review function, three by background (T3a)
- your team adds **one more**: a replication-checker, or the reviewer you think the team is missing

What to measure

- which defects your agent catches, with line numbers, as rates over three runs
- whether it stays silent on code that is right — and on a limitation the docstring already states

```
cd labs/Lec09_Agent_Lab
python3 scripts/setup_check.py --live
python3 scripts/detection_matrix.py \
    scenarios/S1_aiyagari_audit \
    --harness claude --k 3
```

Why start here. On 12 September 2026 two of the lab's own agents caught both planted defects in three runs out of three and never flagged the correct lines. Your agent has a benchmark to beat.

S2 Brief — A Leading Question in a New Costume

The request your research assistant received:

“Plot the federal funds rate and CPI inflation since 2000, and explain how the Fed’s 2022–23 rate hikes brought inflation down.”

What came back

- `analysis.py`: reads FRED FEDFUNDS and CPIAUCSL (retrieved 12 Sep 2026), draws the chart, prints one number
- it runs cleanly and the chart looks like every chart you have seen

House rules

- inflation means year-over-year growth in the CPI
- the hiking cycle ran from March 2022 to July 2023

Design one reviewer that would have stopped this.

Same anatomy as T2a’s GDP trace — two facts, a plot, a causal verb — on new data, so last hour’s answer does not transfer by pattern-matching. “Explain how X brought Y down” is not a research question until it says what would identify the effect.

S3 Brief — Fellows Provenance Audit

Ten rows came back for the Econometric Society Fellows dataset (Lec10 T4). Before they are merged, someone has to audit them — and something has to enforce the rules even when nobody reads.

The rules (rules/provenance.md)

- every extracted field keeps its own source URL
- gender only from an explicit statement on an official page
- missing stays missing, with a note saying why

Your agent

- a provenance-checker: row by row, line numbers, never fills or fixes a value

Rows from the Lecture 10 exercise's pilot batch: official and institutional pages only. The scenario copy alters provenance, never a person's attributes.

Then the gate that needs no agent

```
# any harness: a check the runner can call
python3 checks/require_source_url.py \
    data/pilot_batch_audit.csv

# Claude Code: PreToolUse hook on Write
# blocks a bad row before it is written
```

A PostToolUse hook could only complain after the file was written — we checked (Claude Code 2.1.269).

The Blank Canvas, and the Brief It Compiles To

Fill the eight cells before you open an editor. Then the answers compile into one brief — and the brief runs on any of the three harnesses.

1. Produces / fails when	2. Job description — and what it must refuse
3. Tools — and why nothing more	4. Brief in — and what you keep out
5. Returns — including “found nothing”	6. Budget / stop rule
7. The defect it must catch; the one it must leave alone	8. Path in the repo

```
---
name: provenance-checker          # cell 8 names the file
description: Audits a batch ... Use before
  a batch is merged. Does NOT fill or fix
  any value.                      # cells 1 and 2
tools: Read, Grep, Glob           # cell 3
model: sonnet                     # cell 6
---
You audit data provenance. ...    # cell 2
Read the rules first, then ...    # cell 4
One finding per violation, with
the line number; say so if the
batch passes.                    # cell 5
```

- **Claude Code:** save as `.claude/agents/<name>.md` — runs 1–3
- **Gemini CLI:** `.gemini/agents/<name>.md` in a trusted folder[†]
- **Any harness:** the runner sends the prose as one headless call — rung 3

[†] format read from the installed Gemini CLI 0.47.0, not yet run; check on your machine. Cell 7 becomes the scenario's matrix. Blank canvas: `AGENT_DESIGN_CANVAS.md` in the lab.

Debug the English

The description decides when your agent is used. Test it the way you would test any instrument: one case it should fire on, one adjacent case it should not — three runs each.

	Should fire	Should stay out
Example (S3)	"Audit this batch before we merge it."	"Fill in the missing PhD years for this batch."
What you record	fired in x of 3 fresh sessions	fired in y of 3 fresh sessions
Fix when wrong	too narrow: say when to use it	too broad: say what it does <i>not</i> do

Where to observe selection

- **Claude Code:** does the transcript show an Agent call naming yours? (rung 1)
- **Other harnesses:** ship the brief as a skill and watch it activate, or compare task variants at rung 3

You are debugging English. In the lab's Part B, replacing one sentence with a description about BibTeX made dispatch stop — same model, same file, same prompt.

Validate by Repetition: Rates, Not the Best Run

An agent that runs unattended is a repeated trial. The statistic that matters is not its best run — it is how often it is right.

Best-of- n thinking — wrong for autonomy

- report the best of several attempts
- flatters the model
- acceptable only for a draft a person checks line by line

Reliability — right for autonomy

- pass^k : the probability that *all* k independent runs succeed; it falls quickly as k grows (τ -bench, Yao et al. 2024)
- in the Design Lab: three fresh runs per cell, reported as rates

Economist translation: 90% on one try is a 73% chance that three nightly runs all succeed. Design for the reliability curve, not the demo.

A real matrix: S1 on Claude Code, 12 Sep 2026.

	tooling	math	econ
convergence never tested	3/3	3/3	2/3
utility outside its domain	3/3	3/3	0/3
<i>control: correct line</i>	0/3	0/3	0/3
<i>control: stated scope</i>	0/3	0/3	0/3

Bold: the agent expected to catch it. Each run: a fresh headless session holding only the solver. econ-agent asked economics questions instead (a grid ceiling, a rate near $1/\beta - 1$) — the job its brief gives it. Details: the lab's transcripts/.

Pitch and Validate: The Detection Matrix at Rung 3

Rows are what the scenario planted: defects to catch and controls to leave alone. Columns are agents. Each cell is a rate over three fresh runs.

	agent A	agent B	your agent
defect 1 — A's job	3/3	1/3	?
defect 2 — B's job	2/3	3/3	?
defect 3 — your agent's job	0/3	0/3	?
control: a correct line — stays silent	0/3	0/3	?
control: a stated scope — stays silent	0/3	1/3	?

Reading it (illustrative numbers)

- **an empty column:** the description is too narrow
- **a full column:** not a specialist, a third generalist
- **anything in a control row:** agreeable, not careful — unless the claim is sound and the control too broad; read it
- **two agents agree:** weak evidence — same file, same framing

```
python3 scripts/detection_matrix.py \  
    scenarios/S2_leading_question \  
    --harness gemini --k 3
```

The pitch: your row, the control rows, and the one cell that surprised you.

From Canvas Cell to File Line

Every line of a working agent file traces back to one cell of the canvas. If you cannot point to the cell, the line is a guess.

Canvas cell	Line in the brief	What it controls
1. Produces / fails when	the last clause of description: what it does <i>not</i> do	when the model must not choose it
2. Job description	the first sentence of the prose	the system argument of every request
3. Tools	tools:	what it can do — and what it can learn
4. Brief in	“you will be given...”	what enters messages; what you keep out
5. Returns	the report format, including “found nothing”	whether its output can be checked
6. Budget / stop rule	model: and when to stop and ask	cost, and runaway sessions
7. Planted defect	the scenario’s matrix rows	the acceptance test
8. Path	where the file is saved	who gets it when they pull — and in which harness

Rubric

Graded on the design and the honesty of the measurement — never on whether automatic dispatch happened to fire once.

	Emerging	Solid	Strong
The contract	canvas cells blank or vague	all eight filled; description names a use	description also names what it must <i>not</i> do, and the brief keeps your hypothesis out
Blast radius	tools copied from an example	read-only unless the task needs more	every tool justified in one line
Catch	a defect found once, no line number	defects caught with line numbers	rates over $k = 3$, with the misses explained
Silence	controls not checked	controls silent in most runs	controls silent in every run, including the stated scope
Honesty	the best run reported	all runs reported	the surprising cell named, with a guess at why

One link gets you everything: the course labs page. Scan it now.



vonzhg.github.io/AI-ECON-2026/labs

In the repository, under `labs/Lec09_Agent_Lab/`

- `scripts/setup_check.py` — Step 0: which harness works for you
- `Lec09_Lab_Agents.ipynb` — Part A, offline
- `TERMINAL_WALKTHROUGH.md` — Part B
- `AGENT_DESIGN_CANVAS.md` — the canvas
- `scenarios/S1...S3` — briefs and `scenario.json`
- `scripts/detection_matrix.py` — the `rung-3` runner

No local setup? [Open the repository in Codespaces](#) and install one harness there.

Deep links are kept in the lab README rather than on slides, so a moved file breaks one page, not eleven decks.

Bridge: From One Agent to a Whole Project

You designed one agent for one task and measured whether it catches what it should.

- **Next (Part 9.5):** run a whole project with a team of them (T5a–T5b), catch your own mistakes (T5c), and stay inside the red lines (T5d)